

Research Report: Simulating Noise and Adversarial Attacks in Tracking Control Neural Network Systems

Date: 2026-02-14

Researcher: Ablasse Kingcaid-Ouedraogo

Project: CAHSI REU -- Defending Tracking Control Neural Networks Against Adversarial Attacks

Query: Common avenues and techniques for simulating noise and adversarial attacks in tracking control neural network projects

Executive Summary

This report provides a comprehensive survey of noise models, adversarial attack techniques, simulation frameworks, benchmark problems, and recent research (2023--2026) relevant to defending neural network-based tracking controllers against adversarial attacks. The research covers discrete-time affine-in-the-inputs systems of the form

$x(k+1) = f(x(k)) + g(x(k))u(k)$, where neural network controllers compute optimal control inputs and adversarial attacks target sensor measurements, actuator signals, or the neural network itself. Key findings include: (1) a well-established taxonomy of noise and attack models ranging from Gaussian white noise to sophisticated false data injection and replay attacks; (2) mature simulation toolkits in MATLAB (Robust Control Toolbox, Deep Learning Toolbox, NNV) and Python (ART, Foolbox, python-control); (3) emerging research at the intersection of reinforcement learning, adversarial training, and control-theoretic stability guarantees; and (4) practical IDS/ACC benchmarks directly relevant to this project's analogy framework.

1. Noise and Disturbance Models for Control Systems

1.1 Gaussian White Noise

Definition: A stationary stochastic process with flat power spectral density and zero mean. In discrete-time:

$$w(k) \sim \mathcal{N}(0, Q)$$

where Q is the covariance matrix (noise intensity).

Application in control systems: Used as the canonical disturbance model in Kalman filtering, LQG control, and stochastic optimal control. Process noise $w(k)$ enters the state equation as:

$$x(k+1) = f(x(k)) + g(x(k))u(k) + w(k)$$

Simulation in MATLAB:

```
% Generate white Gaussian noise
w = sqrt(Q) * randn(n, N); % n states, N time steps

% Using idinput for system identification
u = idinput(N, 'rgs'); % Random Gaussian signal
```

Simulation in Python:

```
import control as ct
import numpy as np

Q = np.array([[0.1]]) # Noise intensity
timepts = np.linspace(0, 10, 1000)
V = ct.white_noise(timepts, Q) # Continuous-time
V_dt = ct.white_noise(timepts, Q, dt=0.01) # Discrete-time
```

1.2 Colored Noise (Ornstein-Uhlenbeck Process)

Definition: Noise with non-flat power spectral density, modeling correlated disturbances. The Ornstein-Uhlenbeck (OU) process is the most common model:

$$dv = -\frac{1}{\tau}v dt + \sigma\sqrt{\frac{2}{\tau}} dW$$

where τ is the correlation time and σ is the noise intensity.

Discrete-time approximation:

$$v(k+1) = e^{-\Delta t/\tau}v(k) + \sigma\sqrt{1 - e^{-2\Delta t/\tau}} \cdot \eta(k), \quad \eta(k) \sim \mathcal{N}(0, 1)$$

Application: Models realistic sensor noise (accelerometers, gyroscopes), wind disturbances, and low-frequency process drift. More realistic than pure white noise for physical systems.

Simulation in MATLAB:

```
% Colored noise via filtering white noise
[b, a] = butter(2, 0.1); % Low-pass Butterworth filter
colored = filter(b, a, randn(1, N));
```

```
% Band-Limited White Noise Simulink block
% Parameters: Noise power, Sample time, Seed
```

1.3 Band-Limited White Noise

Any practical realization of white noise is band-limited. In MATLAB/Simulink, the **Band-Limited White Noise** block generates noise that appears "white" within a specified frequency band. Key parameters:

- Noise power (variance)
- Sample time
- Seed for reproducibility

1.4 Bounded Disturbances

Definition: Disturbances constrained within known bounds: $w(k) \in \mathcal{W} = [-\bar{w}, \bar{w}]$

Application: Used in robust control, set-based methods, and reachability analysis. Particularly relevant for worst-case adversarial analysis where the attacker's power is bounded.

Simulation:

```
# Uniform bounded disturbance
w = np.random.uniform(-w_bar, w_bar, size=(n, N))

# L-infinity bounded perturbation (as in FGSM/PGD)
delta = np.clip(perturbation, -epsilon, epsilon)
```

1.5 Pseudo-Random Binary Sequences (PRBS)

Definition: A periodic, deterministic signal with white-noise-like properties that shifts between two values. Maximum period length: $2^n - 1$ where n is the PRBS order.

Application: System identification, frequency response estimation, and -- critically for this project -- simulating adversarial on/off injection attacks.

Simulation in MATLAB:

```
% Generate PRBS signal
u = idinput([100, 1, 5], 'prbs', [0 1], [-2, 2]);

% Using PRBS Signal Generator block in Simulink
% Parameters: Order n, Sample time, Amplitude range
```

```
% Using frest.PRBS for frequency response estimation
input = frest.PRBS('Order', 10, 'Amplitude', 1, 'NumPeriods', 3);
```

1.6 Noise Injection Points

Injection Point	Model	Physical Meaning
Process noise	$x(k+1) = f(x) + g(x)u + w(k)$	Unmodeled dynamics, environmental disturbances
Measurement noise	$y(k) = h(x(k)) + v(k)$	Sensor imprecision, quantization errors
Actuator noise	$u_{actual}(k) = u_{commanded}(k) + \delta(k)$	Actuator imprecision, communication delays
Input disturbance	$x(k+1) = f(x) + g(x)(u + d(k))$	External forces, unmodeled inputs

2. Adversarial Attack Types on Neural Network Controllers

2.1 Gradient-Based Attacks on the Neural Network

2.1.1 Fast Gradient Sign Method (FGSM)

Formulation:

$$x_{adv} = x + \epsilon \cdot \text{sign}(\nabla_x J(\theta, x, y))$$

where J is the loss function, θ the network parameters, x the input, y the true label, and ϵ the perturbation magnitude.

Application to control: Perturb the state observation $x(k)$ fed to the NN controller to cause suboptimal or destabilizing control actions.

Implementation in MATLAB:

```
% Compute gradient of loss w.r.t. input
gradient = dlfeval(@untargetedGradients, net, X, T);
epsilon = 0.1;
X_adv = X + epsilon * sign(gradient);

function gradient = untargetedGradients(dlnet, X, target)
```

```

Y = predict(dlnet, X);
loss = crossentropy(Y, target);
gradient = dlgradient(loss, X);
end

```

Implementation in Python (ART):

```

from art.attacks.evasion import FastGradientMethod
from art.estimators.classification import PyTorchClassifier

classifier = PyTorchClassifier(model=nn_controller, loss=loss_fn,
                              optimizer=optimizer, input_shape=(n,),
                              nb_classes=num_actions)
attack = FastGradientMethod(estimator=classifier, eps=0.1)
x_adv = attack.generate(x=state_observations)

```

Implementation in Python (Foolbox):

```

import foolbox as fb

fmodel = fb.PyTorchModel(nn_controller, bounds=(x_min, x_max))
attack = fb.attacks.FGSM()
_, x_adv, _ = attack(fmodel, states, labels, epsilons=[0.01, 0.05, 0.1])

```

2.1.2 Projected Gradient Descent (PGD)

Formulation (iterative):

$$x_{adv}^{(t+1)} = \Pi_{x+\mathcal{S}} \left(x_{adv}^{(t)} + \alpha \cdot \text{sign}(\nabla_x J(\theta, x_{adv}^{(t)}, y)) \right)$$

where Π denotes projection onto the ϵ -ball $\mathcal{S}$ and α is the step size.

Advantages over FGSM: Multi-step refinement produces stronger adversarial examples. PGD is considered the strongest first-order attack.

Implementation in MATLAB (BIM/PGD):

```

epsilon = 5; alpha = 0.2; numIter = 25;
delta = zeros(size(X), like=X);
for i = 1:numIter
    gradient = dlfeval(@targetedGradients, net, X+delta, target);
    delta = delta - alpha * sign(gradient);
    delta(delta > epsilon) = epsilon;
    delta(delta < -epsilon) = -epsilon;
end

```

```
end
X_adv = X + delta;
```

Implementation in Python (ART):

```
from art.attacks.evasion import ProjectedGradientDescent

attack = ProjectedGradientDescent(estimator=classifier,
                                  eps=0.3, eps_step=0.01,
                                  max_iter=40, targeted=False)
x_adv = attack.generate(x=state_observations)
```

2.1.3 DeepFool

Computes the minimal perturbation to cross the decision boundary by linearizing the classifier at each iteration. Produces smaller perturbations than FGSM/PGD.

2.1.4 Carlini & Wagner (C&W)

Optimization-based attack minimizing perturbation subject to misclassification:

$$\min \|x_{adv} - x\|_p + c \cdot f(x_{adv})$$

Considered one of the most powerful attacks, but computationally expensive.

2.2 Attacks on System Signals

2.2.1 Sensor Spoofing Attacks

Model:

$$y_{compromised}(k) = y_{true}(k) + a_s(k)$$

where $a_s(k)$ is the spoofing signal injected into sensor measurements.

ACC Context: An attacker spoofs the radar/lidar distance measurement, causing the ACC to perceive incorrect inter-vehicle distance. This can lead to unsafe following distances or unnecessary braking.

Simulation approach:

```
# Sensor spoofing on position measurement
def spoofed_measurement(y_true, attack_magnitude, attack_start, k):
    if k >= attack_start:
        # Gradual ramp attack (stealthy)
        a_s = attack_magnitude * (1 - np.exp(-0.5 * (k - attack_start)))
```

```

    return y_true + a_s
return y_true

```

MATLAB (Simulink):

```

% Using the MathWorks vehicle platooning example
enableAttack = 1;
% Attack injects oscillatory signal into velocity information
% at t = 15s in V2V communication channel

```

2.2.2 Actuator Injection Attacks

Model:

$$u_{actual}(k) = u_{NN}(k) + a_u(k)$$

where $u_{NN}(k)$ is the NN controller output and $a_u(k)$ is the injected attack signal.

Simulation:

```

% Actuator attack: add adversarial signal to control input
u_actual = u_nn + attack_amplitude * sin(2*pi*f_attack*t);

```

2.2.3 False Data Injection (FDI) Attacks

Model:

$$y_{attacked}(k) = \phi(x(k), f_1(k)) = Cx(k) + f(k)$$

where $f(k) = \alpha(k) + \theta(k)$ represents the malicious injection plus measurement noise.

Types tested in recent research (PMC 12197249):

1. **Abrupt:** Step injection of magnitude 0.2 at $t = 0$
2. **Incipient:** Gradual bias growth: $0.2(1 - e^{-0.5t})$
3. **Triangle:** Linearly increasing then decreasing: $0.3846(t - 12)$

Key assumption: The attack signal satisfies $\|f(t)\| \leq \bar{f}$ (bounded and continuous).

Detection via Kalman filter residual:

```

% Kalman filter residual-based detection
residual = y_observed - y_predicted;
% Average residuals over time window
avg_residual = movmean(abs(residual), window_size);

```

```
% Detection threshold (calibrated from baseline)
attack_detected = avg_residual > threshold;
```

2.2.4 Replay Attacks

Model: The attacker records legitimate sensor data $\{y(k)\}_{k=t_0}^{t_1}$ and replays it:

$$y_{replayed}(k) = y_{recorded}(k - \Delta)$$

Detection approaches (from recent literature):

- Data-driven detection using moving window subspace identification to construct linear discrete time-varying models
- Output coding strategy transforms replay attacks into detectable additive attacks
- Time-varying H_∞ filter with detection functions based on output residuals
- Deep reinforcement learning-based detection (model-free approach)

Simulation:

```
# Replay attack simulation
def replay_attack(y_buffer, record_start, record_end, replay_start, k):
    if k >= replay_start:
        # Replay recorded data
        replay_idx = (k - replay_start) % (record_end - record_start)
        return y_buffer[record_start + replay_idx]
    return y_current[k]
```

2.2.5 Man-in-the-Middle (MITM) Attacks

Context: In CAN bus and V2V communication, MITM attacks intercept and modify messages between controllers and sensors/actuators.

Detection approaches:

- Deep Convolutional Neural Networks (DCNN) for CAN bus intrusion detection
- LSTM-based sequential pattern detection (98.1% accuracy, 97.9% F1-score)
- Multi-stage IDS: ANN for known attacks + LSTM autoencoder for unknown attacks
- GAN-based anomaly detection using vehicle trajectory data

Simulation datasets:

- Car Hacking Dataset
- OTIDS (Offender Threat Identification Dataset)
- SWaT (Secure Water Treatment)

- WADI (Water Distribution)

2.3 Attack Taxonomy Summary

Attack Type	Target	Knowledge Required	Stealthiness	Implementation Complexity
FGSM	NN input/state	White-box (gradients)	Low	Low
PGD	NN input/state	White-box (gradients)	Medium	Medium
C&W	NN input/state	White-box (full model)	High	High
DeepFool	NN input/state	White-box (gradients)	High	Medium
Sensor spoofing	Measurement channel	Black-box (physical)	Variable	Low-Medium
Actuator injection	Control channel	Black-box (physical)	Low	Low
FDI	Sensor/state data	Gray-box (model knowledge)	High	Medium
Replay	Communication channel	Black-box (recording)	High	Low
MITM	Communication bus	Gray-box (protocol)	Variable	Medium

3. Simulation Frameworks and Tools

3.1 MATLAB/Simulink

3.1.1 Control System Toolbox

- `lsim` : Simulate time response with arbitrary inputs (including noise)
- `ss` , `tf` , `zpk` : State-space, transfer function, zero-pole-gain models
- `c2d` : Continuous-to-discrete conversion with various methods (ZOH, Tustin)
- `lqr` , `lqe` : LQR controller and Kalman filter design

3.1.2 Robust Control Toolbox

- `ureal`, `ultidyn` : Uncertain real parameters and unstructured dynamics
- `robstab`, `robgain` : Robust stability and performance analysis
- `wcgain` : Worst-case gain computation
- `musyn` : μ -synthesis for robust controller design

3.1.3 Deep Learning Toolbox

- **FGSM/BIM adversarial example generation** (built-in support)
- `verifyNetworkRobustness` : Formal verification of adversarial robustness
- **Adversarial training**: Train networks robust to FGSM perturbations
- **Deep Learning Toolbox Verification Library**: Additional verification support

3.1.4 Simulink Blocks for Attack Simulation

- **Band-Limited White Noise**: Configurable noise power, sample time, seed
- **Random Number**: Generate Gaussian or uniform random signals
- **PRBS Signal Generator**: Pseudo-random binary sequences for testing
- **Injection Attacks Toolbox** (Potluri et al., IEEE 2020): MATLAB/Simulink toolbox for simulating injection attacks in ICS -- GitHub: [sasankapotluri/ICS-Injection_Attack_Toolbox](https://github.com/sasankapotluri/ICS-Injection_Attack_Toolbox)

3.1.5 System Identification Toolbox

- `idinput` : Generate test signals (RBS, RGS, PRBS, sine)

```
u = idinput(N, 'prbs', [0 B], [-A, A]); % PRBS with band and range
u = idinput(N, 'rgs'); % Random Gaussian signal
u = idinput(N, 'rbs'); % Random binary signal
```

3.1.6 Reinforcement Learning Toolbox

- Train adversarial agents to discover worst-case attack patterns
- Deep Q-Network agents for vulnerability assessment
- Example: Lockheed Martin 5G vulnerability assessment

3.1.7 NNV: Neural Network Verification Tool

- **MATLAB-based** toolbox for formal verification of NN controllers
- **Reachability analysis**: Exact and over-approximate methods
- **Set representations**: Polyhedra, star sets, zonotopes, abstract domains
- **Supported architectures**: FFNNs, CNNs, RNNs, Neural ODEs
- **Version 3.0 features**: Probabilistic verification, weight perturbation analysis

- **Installation:** `git clone --recursive https://github.com/verivital/nnv.git`
- **Requires:** MATLAB 2023a+, multiple toolboxes

3.1.8 Vehicle Platooning Attack Example (MathWorks)

Complete Simulink example for detecting/mitigating FDI attacks in V2V platooning:

```
% Three modes of operation:
enableAttack = 1; enableAttackDetection = 0; % Attack only
enableAttack = 0; enableAttackDetection = 1; % Calibrate threshold
enableAttack = 1; enableAttackDetection = 1; % Full detection
% Detection: Kalman filter residual > 0.1 after 5s settling
% Mitigation: Switch from platooning to ACC mode
```

3.2 Python Libraries

3.2.1 IBM Adversarial Robustness Toolbox (ART)

- **Version:** 1.17.0 | **License:** MIT | **Stars:** 5.8k
- **Supported frameworks:** TensorFlow, Keras, PyTorch, scikit-learn, XGBoost, etc.
- **Evasion attacks:** FGSM, PGD, DeepFool, C&W (L0/L2/Linf), AutoAttack, Boundary Attack, JSMA, Square Attack, ZOO, Adversarial Patch
- **Defenses:** Adversarial training (standard, Madry PGD, TRADES), feature squeezing, Gaussian augmentation, JPEG compression, input/activation detectors
- **Installation:** `pip install adversarial-robustness-toolbox`
- **GitHub:** `Trusted-AI/adversarial-robustness-toolbox`

```
from art.attacks.evasion import FastGradientMethod, ProjectedGradientDescent
from art.defences.trainer import AdversarialTrainer

# Create attack
fgsm = FastGradientMethod(estimator=classifier, eps=0.1)
pgd = ProjectedGradientDescent(estimator=classifier, eps=0.3,
                               eps_step=0.01, max_iter=40)

# Generate adversarial examples
x_adv_fgsm = fgsm.generate(x=x_test)
x_adv_pgd = pgd.generate(x=x_test)

# Adversarial training defense
trainer = AdversarialTrainer(classifier, attacks=fgsm, ratio=0.5)
trainer.fit(x_train, y_train, nb_epochs=50)
```

3.2.2 Foolbox

- **Version:** 3.x | Native support for PyTorch, TensorFlow, JAX
- **Key attacks:** FGSM, PGD, DeepFool, C&W, Boundary Attack, Brendel & Bethge
- **Installation:** `pip install foolbox`

```
import foolbox as fb

fmodel = fb.PyTorchModel(model, bounds=(0, 1))
attack = fb.attacks.LinfPGD()
_, adv_examples, success = attack(fmodel, inputs, labels, epsilons=0.1)
```

3.2.3 CleverHans

- **Version:** 4.0+ | Supports JAX, PyTorch, TF2
- **Focus:** Reference implementations for benchmarking
- **GitHub:** `cleverhans-lab/cleverhans`

3.2.4 Python Control Systems Library

- **Version:** 0.10.2
- **Key functions for noise simulation:**
 - `ct.white_noise(timepts, Q, dt)` : Generate white noise
 - `ct.correlation(timepts, X, Y)` : Compute correlation matrices
 - `ct.lqe(sys, QN, RN)` : Kalman filter design
 - `ct.create_estimator_iosystem(sys, Qv, Qw)` : Estimator implementation
 - `ct.forced_response(sys, timepts, V)` : Simulate with noise input
 - `ct.input_output_response(sys, T, U)` : Nonlinear system simulation

```
import control as ct
import numpy as np

# Discrete-time nonlinear system with noise
def plant_update(t, x, u, params):
    f_x = np.array([x[0] + 0.1*x[1], x[1] - 0.1*np.sin(x[0])])
    g_x = np.array([0, 0.1])
    w = np.random.normal(0, params.get('noise_std', 0.01), size=x.shape)
    return f_x + g_x * u + w

def plant_output(t, x, u, params):
    v = np.random.normal(0, params.get('meas_noise', 0.005), size=x.shape)
    return x + v # measurement noise
```

```
plant = ct.nlsys(plant_update, plant_output, states=2, inputs=1, outputs=2,
                dt=0.01, name='plant')
```

3.2.5 OpenAI Gymnasium

- Standard RL environments for control benchmarks
- **Classic control**: CartPole, Pendulum, Acrobot, MountainCar
- **Continuous control**: HalfCheetah, Hopper, Walker2d (MuJoCo)
- Useful for testing adversarial perturbations on RL-based controllers

```
import gymnasium as gym

env = gym.make('Pendulum-v1')
# Add adversarial perturbation wrapper
class AdversarialWrapper(gym.Wrapper):
    def __init__(self, env, epsilon=0.1):
        super().__init__(env)
        self.epsilon = epsilon

    def step(self, action):
        # Inject adversarial perturbation to observation
        obs, reward, terminated, truncated, info = self.env.step(action)
        perturbation = np.random.uniform(-self.epsilon, self.epsilon,
        obs.shape)
        obs_adv = obs + perturbation
        return obs_adv, reward, terminated, truncated, info
```

3.2.6 RobustBench

- Standardized adversarial robustness benchmarking
- Uses AutoAttack (ensemble of white-box and black-box attacks)
- Leaderboards for model robustness evaluation
- **GitHub**: [RobustBench/robustbench](https://github.com/RobustBench/robustbench)

3.3 C++ Libraries

3.3.1 Eigen

- High-performance matrix operations for control system simulation
- Linear algebra backend for state-space computations
- Header-only library, easy integration

```
#include <Eigen/Dense>

// Discrete-time state update with noise
Eigen::VectorXd state_update(const Eigen::VectorXd& x,
                             const Eigen::VectorXd& u,
                             const Eigen::VectorXd& w) {
    Eigen::VectorXd f_x = /* nonlinear dynamics */;
    Eigen::MatrixXd g_x = /* input matrix */;
    return f_x + g_x * u + w;
}
```

3.3.2 LibTorch (PyTorch C++ Frontend)

- C++ interface to PyTorch for NN inference
- TorchScript serialization for model deployment
- Gradient computation for implementing FGSM/PGD in C++

```
#include <torch/torch.h>

// Load trained NN controller
torch::jit::script::Module controller = torch::jit::load("controller.pt");

// FGSM attack in C++
torch::Tensor fgsm_attack(torch::jit::script::Module& model,
                           torch::Tensor input, torch::Tensor target,
                           float epsilon) {
    input.set_requires_grad(true);
    auto output = model.forward({input}).toTensor();
    auto loss = torch::nn::functional::mse_loss(output, target);
    loss.backward();
    auto perturbation = epsilon * input.grad().sign();
    return input + perturbation;
}
```

3.3.3 ONNX Runtime

- Cross-platform NN inference engine
 - Export PyTorch/TensorFlow models to ONNX format
 - Hardware acceleration (CUDA, TensorRT, OpenVINO, CoreML)
 - C++ API for embedded/real-time applications
-

4. Benchmark Problems and Test Scenarios

4.1 Control System Benchmarks

Benchmark	System Type	State Dim	Why Relevant
Inverted Pendulum	Nonlinear, underactuated	4	Classic NN controller test
Cart-Pole	Nonlinear, discrete actions	4	Standard RL benchmark
ACC (Adaptive Cruise Control)	Nonlinear, tracking	3-6	Direct project relevance
Vehicle Platooning	Multi-agent, networked	N*3	V2V attack scenarios
2-DOF Robot Manipulator	Nonlinear, multi-input	4	Tracking control benchmark
Dubin's Car	Nonlinear, kinematic	3	Path tracking with disturbances
Tennessee Eastman Process	Industrial process	52	Comprehensive attack benchmark

4.2 ACC/IDS Benchmark (Directly Relevant)

The ACC system provides an ideal benchmark for this research:

System dynamics:

$$\dot{v}_{ego} = \frac{1}{m}(F_{engine} - F_{brake} - F_{drag})$$
$$\dot{d}_{rel} = v_{lead} - v_{ego}$$

Attack scenarios:

- Speed spoofing:** Attacker modifies perceived lead vehicle speed
- Distance injection:** False radar/lidar distance measurements
- Acceleration manipulation:** Modify commanded acceleration
- V2V message tampering:** Alter cooperative awareness messages

Metrics for evaluation:

- Tracking error (RMSE of velocity/distance)

- Control effort (integral of squared control)
- Time-to-detect (attack detection latency)
- False positive rate / false negative rate
- Safety margin violation frequency
- Attack estimation accuracy (RMSE of attack signal estimate)

4.3 Recent Benchmark Results

Tennessee Eastman Process (TEP) benchmark (arXiv 2403.13502):

- 52 sensor readings, 2000 timestamps, 28 fault types
- Models tested: MLP (3.4M params), GRU (205K params), TCN (152K params)
- Attack parameters: ϵ range 0-0.3 in 0.015 increments
- Six attack types tested: Random, FGSM, PGD, DeepFool, C&W, FGSM-distillation
- Best defense: Combined adversarial training + quantization (2^5 levels)

RobustBench standardized benchmark:

- AutoAttack ensemble for evaluation
- CIFAR-10-C for common corruption robustness
- Tracks state-of-the-art adversarial robustness across model architectures

4.4 Evaluation Metrics

Metric	Formula/Description	Use Case
Tracking RMSE	$\sqrt{\frac{1}{N} \sum (x_{ref}(k) - x(k))^2}$	Attack impact on tracking
Control effort	$\sum u(k)^2$ or $\int u(t)^2 dt$	Energy cost under attack
Attack detection rate	$TP / (TP + FN)$	Defense effectiveness
False alarm rate	$FP / (FP + TN)$	Detection reliability
Detection delay	$t_{detect} - t_{attack}$	Response time
Attack estimation RMSE	$\sqrt{\frac{1}{N} \sum (\hat{a}(k) - a(k))^2}$	Attack signal reconstruction
Robustness radius	Max ϵ maintaining stability	Certified robustness
Communication reduction	% fewer control transmissions	Event-triggered efficiency

5. Recent Research and Approaches (2023--2026)

5.1 RL-Based Tracking Control with Adversarial Attacks

Key paper: "Reinforcement learning-based optimised control for tracking of nonlinear systems with adversarial attacks" (arXiv:2209.02165)

- **System model:** Nonlinear discrete-time with adversarial attacks on actuators and outputs
- **Approach:** Simultaneously adapts three neural networks: critic, actor, and adversary
- **Method:** Solves the Hamilton-Jacobi-Bellman (HJB) equation via online RL
- **Application:** Robot manipulator tracking control
- **Key insight:** The adversary NN learns worst-case attacks while the actor NN learns to compensate, producing a minimax optimal controller

5.2 Event-Triggered Secure Control Against FDI

Key paper: "Event-Triggered Secure Control Design Against FDI Attacks via Lyapunov-Based Neural Networks" (PMC 12197249)

- **System:** 2-DOF robot manipulator, linearized state-space model
- **Hybrid observer:** Kalman filter + 3-layer feedforward NN (16-5-4 architecture)
- **NN training:** Online Lyapunov-based adaptation (no backpropagation)
- **Weight update laws:** Projection-based bounded adaptation using Lyapunov stability
- **Event-triggering condition:** $\|e_{\hat{x}}(t)\|^2 \geq \sigma \|\hat{x}(t)\|^2$
- **Results:** Attack estimation RMSE as low as 1.36×10^{-4} ; 97%+ communication reduction
- **Attack scenarios tested:** Abrupt (step), incipient (exponential), triangle (ramp)

5.3 Neural Network Safe Tracking Controllers

Key paper: "Learning Neural Network Safe Tracking Controllers from Backward Reachable Sets" (arXiv:2511.21953)

- **System:** Discrete nonlinear: $x_{k+1} \in f(x_k, u_k) + \mathcal{W}$
- **Disturbance model:** Bounded hyper-rectangular sets $\mathcal{W} = [-\bar{w}, \bar{w}]$
- **Method:** Zonotopic backward reachable sets along nominal trajectories
- **Safety guarantees:** 99.9% confidence via conformal prediction
- **NN architecture:** Feedforward with tanh activation, input normalization
- **Key insight:** States within backward reachable sets are guaranteed to satisfy reach-avoid specifications under worst-case disturbances

5.4 Certified Neural Network Control Architectures

Key paper: "Certified Neural Network Control Architectures: Methodological Advances" (MDPI Mathematics 2025)

- **Stability guarantees:** Lyapunov-based methods reconcile data-driven learning with control theory
- **Robustness:** Stochastic barrier functions provide probabilistic safety guarantees
- **Verification tools:** ReachNN for formal verification of closed-loop behaviors
- **Adversarial defense:** Frequency filtering attenuates high-frequency adversarial noise
- **Cross-domain:** Applications in chemical processes, power grids, flow control

5.5 Robust DRL Against Adversarial Attacks

Key survey: "Robust Deep Reinforcement Learning Through Adversarial Attacks and Training" (arXiv:2403.00420, 2024)

- **Attack taxonomy for DRL:**
 - State/observation attacks (perturb what agent sees)
 - Action attacks (modify agent's actions)
 - Reward attacks (corrupt reward signal)
 - Environment dynamics attacks (modify transition function)
- **Optimal attacks:** Derived through polynomial-time planning or learning
- **Defense methods:** Adversarial training, regularization, certified robustness
- **Key finding:** Adversarial manipulation makes true environment state only partially observable

5.6 ACC Anomaly Detection and Mitigation

Key paper: "Machine learning-based detection and mitigation of cyberattacks in adaptive cruise control systems" (Nature Scientific Reports, 2025)

- **System:** ACC-equipped vehicles with V2V communication
- **Attack model:** Manipulation/forgery of V2V messages (speed, distance)
- **Detection:** ACCDM (ACC anomaly Detection and Mitigation) model
- **Approach:** Continuously monitors vehicle parameters, detects deviations
- **GAN-based method:** Anomaly detection using vehicle trajectory data
- **Results:** LSTM achieves 98.1% accuracy, 97.9% F1-score

5.7 Data-Driven Replay Attack Detection

Key paper: "Data-driven replay attack detection for unknown cyber-physical systems" (Information Sciences, 2024)

- **Method:** Moving window subspace identification for linear discrete time-varying models
- **Key technique:** Output coding strategy transforms replay into detectable additive attacks

- **Detection:** Time-varying H_∞ filter with residual-based detection functions
- **Advantage:** Works for unknown CPS (no prior model required)

5.8 NNV Verification for Closed-Loop Systems

Tool: NNV 3.0 (verivital/nnv, MATLAB)

- **Verification approach:** Set-based reachability analysis
 - **Closed-loop support:** Exact and over-approximate reachability for linear plants + FFNN controllers with ReLU activations
 - **New in 3.0:** Probabilistic verification, weight perturbation, time-dependent networks
 - **Alpha-beta-CROWN:** GPU-accelerated NN verifier (winner of VNN-COMP 2021-2025)
-

6. Cross-Reference Analysis

6.1 Consensus Points

1. **FGSM and PGD are the standard baseline attacks** for evaluating NN controller robustness. All sources agree these should be the starting point for any adversarial testing framework.
2. **Kalman filter residual-based detection** is the most widely used method for detecting FDI and sensor spoofing attacks in control systems. Both MATLAB examples and academic papers converge on this approach.
3. **Adversarial training** (training with perturbed data) consistently improves robustness but at the cost of nominal performance degradation. The TEP benchmark study, ART documentation, and MATLAB tutorials all confirm this trade-off.
4. **Bounded disturbance models** are preferred for worst-case analysis and formal verification, while Gaussian models are preferred for probabilistic analysis and Kalman filtering.
5. **Event-triggered control** can simultaneously reduce communication overhead and improve resilience to attacks by limiting the attack surface.

6.2 Contested/Nuanced Points

1. **White-box vs. black-box threat model:** Academic literature often assumes white-box access for stronger theoretical guarantees, but practical attacks on control systems are more likely black-box. The simulation framework should support both.
2. **Stealthiness vs. effectiveness trade-off:** FDI and replay attacks are stealthy but limited in impact; FGSM/PGD are effective but potentially detectable. The choice depends on the assumed attacker model.

3. **Online vs. offline adversarial training:** The Lyapunov-based approach (online, no backpropagation) contrasts with standard adversarial training (offline, backpropagation). The control-theoretic approach provides stability guarantees but may be less flexible.

6.3 Mapping to Project Architecture

For the CAHSI REU project with system $x(k+1) = f(x(k)) + g(x(k))u(k)$:

Project Component	Recommended Approach	Tool
Noise simulation	White noise + PRBS	MATLAB <code>idinput</code> , Python <code>ct.white_noise</code>
Gradient attacks	FGSM, PGD on NN controller	ART, MATLAB Deep Learning Toolbox
Signal attacks	FDI on measurements	Custom MATLAB/Python injection
Attack detection	Kalman filter residual	MATLAB Control Toolbox, python-control
Robustness verification	Reachability analysis	NNV (MATLAB)
Benchmark scenario	ACC tracking control	MATLAB Simulink ACC example
Defense evaluation	Adversarial training + detection	ART + custom metrics

7. Practical Implementation Roadmap

Phase 1: Baseline System Setup

1. Implement discrete-time tracking control system in MATLAB and Python
2. Train NN controller (actor-critic or direct policy) for tracking
3. Validate nominal tracking performance (no attacks/noise)

Phase 2: Noise and Disturbance Testing

1. Add Gaussian white noise (process + measurement) at varying intensities
2. Add colored noise (OU process) for realistic sensor modeling
3. Add PRBS signals for pseudo-random adversarial testing
4. Measure tracking degradation vs. noise intensity

Phase 3: Adversarial Attack Simulation

1. Implement FGSM attacks on NN controller inputs (white-box)
2. Implement PGD attacks for stronger adversarial evaluation
3. Implement FDI attacks on sensor measurements
4. Implement replay attacks on control signals
5. Implement sensor spoofing (ACC-specific)
6. Sweep ϵ from 0 to 0.3 and measure tracking error

Phase 4: Defense Implementation

1. Kalman filter-based attack detection (residual monitoring)
2. Adversarial training of NN controller
3. Event-triggered control for reduced attack surface
4. Hybrid observer (Kalman + NN) for attack estimation

Phase 5: Verification and Benchmarking

1. Formal verification using NNV reachability analysis
 2. Evaluate on ACC benchmark scenario
 3. Compare metrics across attack types and defense mechanisms
 4. Statistical analysis of robustness guarantees
-

8. Key Insights

1. **The RL-based actor-critic-adversary framework** (arXiv:2209.02165) is the most directly relevant approach for this project, as it uses the same system form and simultaneously trains the controller and adversary.
2. **IBM ART is the most comprehensive attack library** for Python-based simulation, supporting 30+ attack types with a unified API across all major ML frameworks. It should be the primary tool for implementing gradient-based attacks.
3. **MATLAB's Deep Learning Toolbox now has built-in FGSM/BIM** adversarial example generation and the `verifyNetworkRobustness` function, enabling tight integration with Simulink control simulations.
4. **PRBS signals serve double duty** -- they are standard for system identification (validating the model) and can simulate binary adversarial injection patterns relevant to the IDS/ACC analogy.
5. **The event-triggered Lyapunov-based NN approach** (PMC 12197249) achieves 97%+ communication reduction while maintaining attack estimation RMSE below 10^{-3} , demonstrating that defense need not come at prohibitive computational cost.

9. Knowledge Gaps

1. **Limited work on adversarial attacks specific to affine-in-the-inputs discrete-time tracking controllers.** Most adversarial ML research focuses on classification; adaptation to control requires careful consideration of temporal dynamics and stability.
 2. **No unified benchmark** exists for evaluating adversarial robustness of NN controllers across different system types. RobustBench covers classifiers; a "ControlRobustBench" would be valuable.
 3. **C++ adversarial attack libraries are underdeveloped.** While LibTorch and ONNX Runtime support inference, there is no C++ equivalent of ART or Foolbox with pre-implemented attacks.
 4. **The interaction between different simultaneous attack types** (e.g., sensor spoofing + actuator injection) is understudied. Most papers consider single attack vectors.
 5. **Transferability of adversarial examples** from one NN controller architecture to another in the control systems domain has not been thoroughly investigated.
-

10. Sources

Academic Papers

- [Adversarial Attacks and Defenses in Automated Control Systems: A Comprehensive Benchmark](#) (arXiv, 2024)
- [RL-based Tracking Control for Nonlinear Systems with Adversarial Attacks](#) (arXiv, 2022)
- [Event-Triggered Secure Control Against FDI Attacks via Lyapunov-Based Neural Networks](#) (PMC, 2025)
- [Learning Neural Network Safe Tracking Controllers from Backward Reachable Sets](#) (arXiv, 2025)
- [Certified Neural Network Control Architectures](#) (MDPI, 2025)
- [Robust Deep Reinforcement Learning Through Adversarial Attacks and Training: A Survey](#) (arXiv, 2024)
- [Data-driven Replay Attack Detection for Unknown CPS](#) (Information Sciences, 2024)
- [Machine Learning-Based Detection of Cyberattacks in ACC Systems](#) (Scientific Reports, 2025)
- [Security Control of CPS under Cyber Attacks: A Survey](#) (PMC, 2024)
- [Reinforcement Learning Solution for CPS Security Against Replay Attacks](#) (IEEE TIFS, 2023)
- [NNV: The Neural Network Verification Tool](#) (GitHub/PMC)

- [Injection Attacks Toolbox in MATLAB/Simulink](#) (IEEE, 2020)
- [Detecting Stealthy Cyberattacks on ACC Vehicles](#) (arXiv, 2023)
- [Resilient Interval Observer-Based Control for Cooperative ACC under FDI Attack](#) (arXiv, 2026)
- [AI-Based IDS for In-Vehicle Networks: A Survey](#) (ACM, 2023)
- [Development of Injection Attacks Toolbox](#) (ResearchGate)

Tools and Documentation

- [IBM Adversarial Robustness Toolbox \(ART\)](#)
- [Foolbox: Adversarial Examples Toolbox](#)
- [CleverHans: Adversarial Example Library](#)
- [Python Control Systems Library - Stochastic Systems](#)
- [Python Control - Discrete Nonlinear Simulation](#)
- [MATLAB idinput Function](#)
- [MATLAB PRBS Input Signals](#)
- [MATLAB Generate Adversarial Examples](#)
- [MATLAB Train Robust Network](#)
- [MATLAB verifyNetworkRobustness](#)
- [MATLAB Detect and Mitigate Attacks in Platooning](#)
- [MATLAB Sensor Spoofing Video](#)
- [RobustBench: Adversarial Robustness Benchmark](#)
- [alpha-beta-CROWN Neural Network Verifier](#)
- [Gymnasium Classic Control Environments](#)
- [TensorFlow FGSM Tutorial](#)
- [PyTorch FGSM Tutorial](#)
- [MIT Underactuated Robotics - Stochastic Systems](#)
- [ONNX Runtime](#)
- [ICS Injection Attack Toolbox](#)

Web Resources

- [Adversarial ML Tutorial](#)
- [MathWorks Adjust Disturbance and Noise Models](#)
- [Engineering LibreTexts - Noise Modeling](#)
- [ScienceDirect - Process Noise Overview](#)